

DOCUMENTO 05 / METODOLOGÍA Y STACK

Cómo se construyó Voxmetriks

Una explicación sencilla de Spec-Driven Development, las herramientas usadas y la forma de comprobar el resultado.

Voxmetriks Studio

Guía de demostración



Una guía breve y visual para explicar el producto con claridad, sin vender humo y sin perder el encanto de la experiencia.

Qué significa Spec-Driven Development

Spec-Driven Development (SDD) significa que la especificación es el punto de partida y la referencia del trabajo. Antes de cambiar una parte importante, se deja claro qué problema se resuelve, qué debe ocurrir, qué queda fuera y cómo se va a comprobar.

En Voxmetriks se usa GitHub Spec Kit con esta cadena:

Constitution -> Specify -> Clarify -> Checklist -> Plan -> Tasks -> Analyze -> Implement -> Validate -> Evidence -> Close

No es burocracia por sí misma. Es una forma de evitar que una idea termine convertida en una pantalla bonita que no cumple el flujo real.

Cómo se aplicó aquí

Etapa	Qué hice	Dónde queda
Constitution	Fijé las reglas del producto, datos y arquitectura.	.specify/memory/constitution.md
Specify	Escribí requisitos y criterios para cada feature no trivial.	.specify/features/
Plan + tasks	Partí el trabajo en cambios pequeños y verificables.	plan.md / tasks.md
Implement	Ajusté Angular, FastAPI, DuckDB y ELT sin reescribir desde cero.	apps/ y analytics/
Validate	Combiné pruebas, lint, build, backend y revisión visual.	tests + evidencia
Evidence + close	Dejé documentación, diagramas, datos demo y commits reproducibles.	docs/ + GitHub

Qué herramientas usé

Cada herramienta tiene un papel concreto. No hay una tecnología puesta solo para llenar la arquitectura.

Capa	Herramienta	Para qué sirve
Interfaz	Angular + TypeScript	Pantallas, rutas, menú, temas y reproductor.
Componentes	Angular Material/CDK + CSS	Controles, accesibilidad y design system ámbar.
Backend	Python + FastAPI + Pydantic	API, reglas, permisos y validación.
Datos	DuckDB	Warehouse local: catálogo, operación y analítica.
Ingesta	ELT + Parquet + Airflow local	Preparar y actualizar el catálogo de forma trazable.
Audio	Spotify SDK + Deezer API	Spotify completo con sesión; Deezer como preview de respaldo.
Seguridad	Sesiones + RBAC	Separar usuarios, organizaciones y permisos.
Calidad	Vitest + pytest + ESLint + Ruff	Detectar regresiones y mantener el código consistente.
Entrega	Git + GitHub	Historial, commits y entrega reproducible en otra laptop.

Decisiones honestas

- YouTube se descartó como fuente de reproducción; Spotify y Deezer cubren el flujo actual.
- DuckDB es el warehouse de esta demo local, no una promesa de base transaccional multiusuario de producción.
- Los datos empresariales de la cuenta admin son sintéticos y están preparados para enseñar el producto.

Cómo sé que funciona

La validación no se quedó en 'abre la pantalla'. Revisé el producto desde cuatro ángulos para poder defenderlo con tranquilidad.

Tipo	Qué se comprobó
Visual	Dark y light, contraste, contornos, menú, tablas, cards, reportes, responsive y reproductor fijo.
Funcional	Login, navegación personal/empresa, búsqueda, cola, Spotify/Deezer, auto-skip, permisos y reportes.
Automática	Tests del frontend y backend, lint y build para detectar regresiones antes de entregar.
Evidencia	Screenshots, logs, datos demo, documentación y commits que permiten repetir la entrega.

Mi respuesta corta

'Primero describí el comportamiento en una spec; después hice el plan y las tareas, lo implementé por partes y lo validé con pruebas y revisión visual. Angular muestra la experiencia, FastAPI aplica las reglas, DuckDB guarda el catálogo y la analítica, y Spotify/Deezer resuelven el audio. Cada cierre queda respaldado por evidencia y por un commit.'

Lo que sí y lo que no estoy diciendo

- Sí: es una demo académica completa, trazable y reproducible.
- Sí: puedo explicar qué hace cada capa y por qué existe.
- No: no la presento como streaming comercial licenciado ni como facturación real.

